

Buffer Pool i Caching mehanizmi

Praktično testiranje u PostgreSQL-u

Student: Katarina Adamović | br. ind. 2173
Predmet: Sistemi za upravljanje bazama podataka
Modul: Bezbednost računarskih sistema
Mentor: Prof. Aleksandar Stanimirović

Sadržaj prezentacije

01

Uvod — zašto je disk I/O problem

Razlika u brzini: RAM vs disk

02

Storage Engine i arhitektura

Slojevi DBMS-a, heap model, stranice

03

Buffer Pool

Koncepti, cache hit/miss, memorijska hijerarhija

04

Životni ciklus stranice

Cache lookup, pin, dirty page, eviction

05

Cold vs Warm cache

LRU, DISCARD ALL, stanje keša

06

Caching mehanizmi

Hijerarhija, Plan Cache, pg_prewarm, OLTP vs OLAP

07

Testno okruženje + Rezultati

Mala i velika baza, Seq Scan, Bitmap, selektivnost

08

Velicina tabele vs shared_buffers

Kada buffer pool postaje ključan faktor

09

Zaključak

Pitanja?

Uvod — Zašto je disk I/O problem?

Sistemi baza podataka se suočavaju sa fundamentalnom razlikom u brzini između RAM-a i diska. Svaki upit koji ne nađe podatke u memoriji mora posegnuti za diskom — a to je skupa operacija.

Latencija po nivou memorije

CPU Keš (L1/L2/L3)

~1 ns | 10.000× brže od HDD

RAM memorija

~100 ns | 1.000× brže od HDD

SSD disk (NVMe)

~100 μs | 10× brže od HDD

HDD disk (magnetni)

~10 ms | 1× (osnova) brže od HDD

100.000×

brži RAM od HDD

8 KB

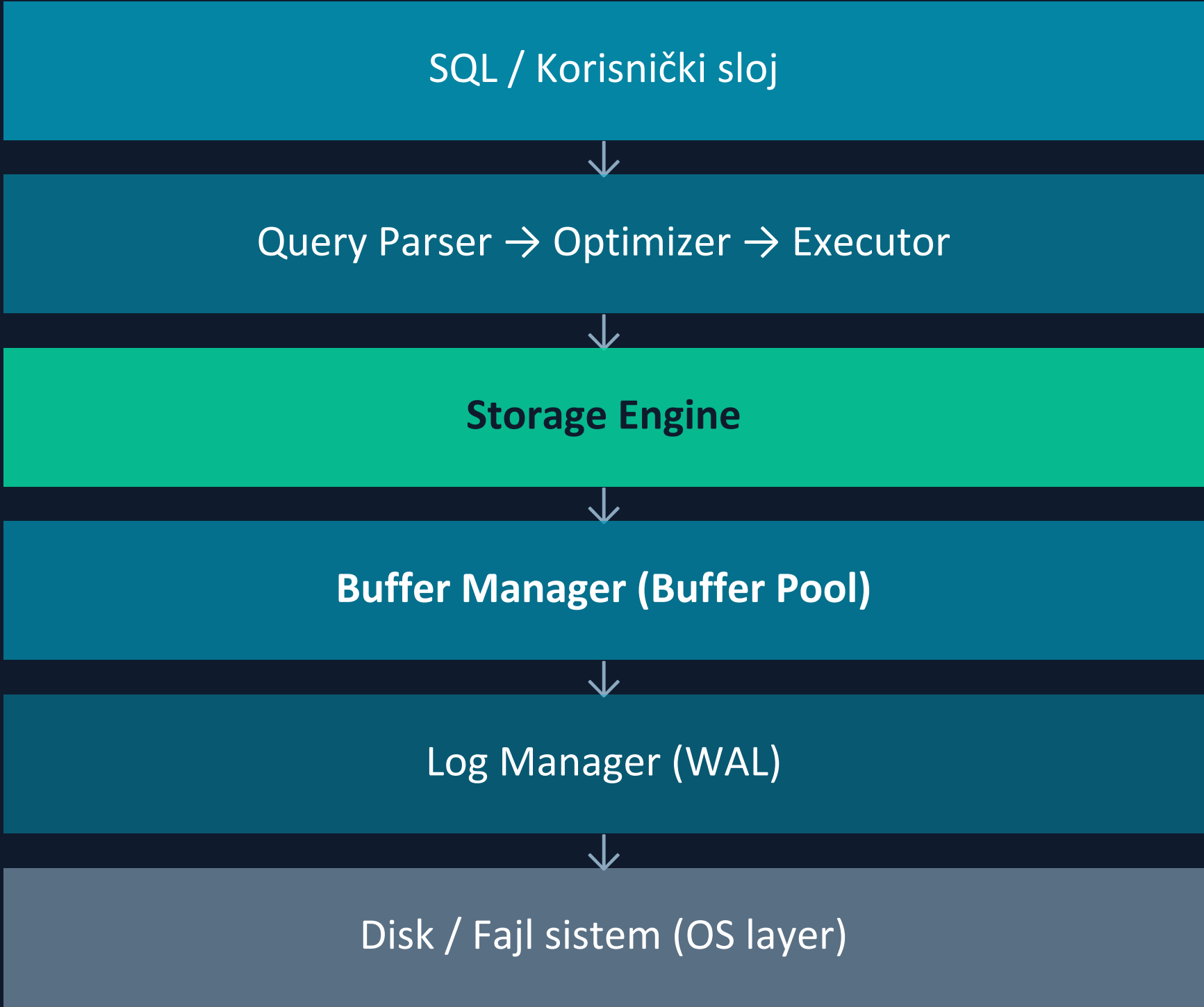
veličina jedne stranice
— uvek se čita cela

I/O

najskuplji deo
izvršavanja upita

Storage Engine i fizička organizacija podataka

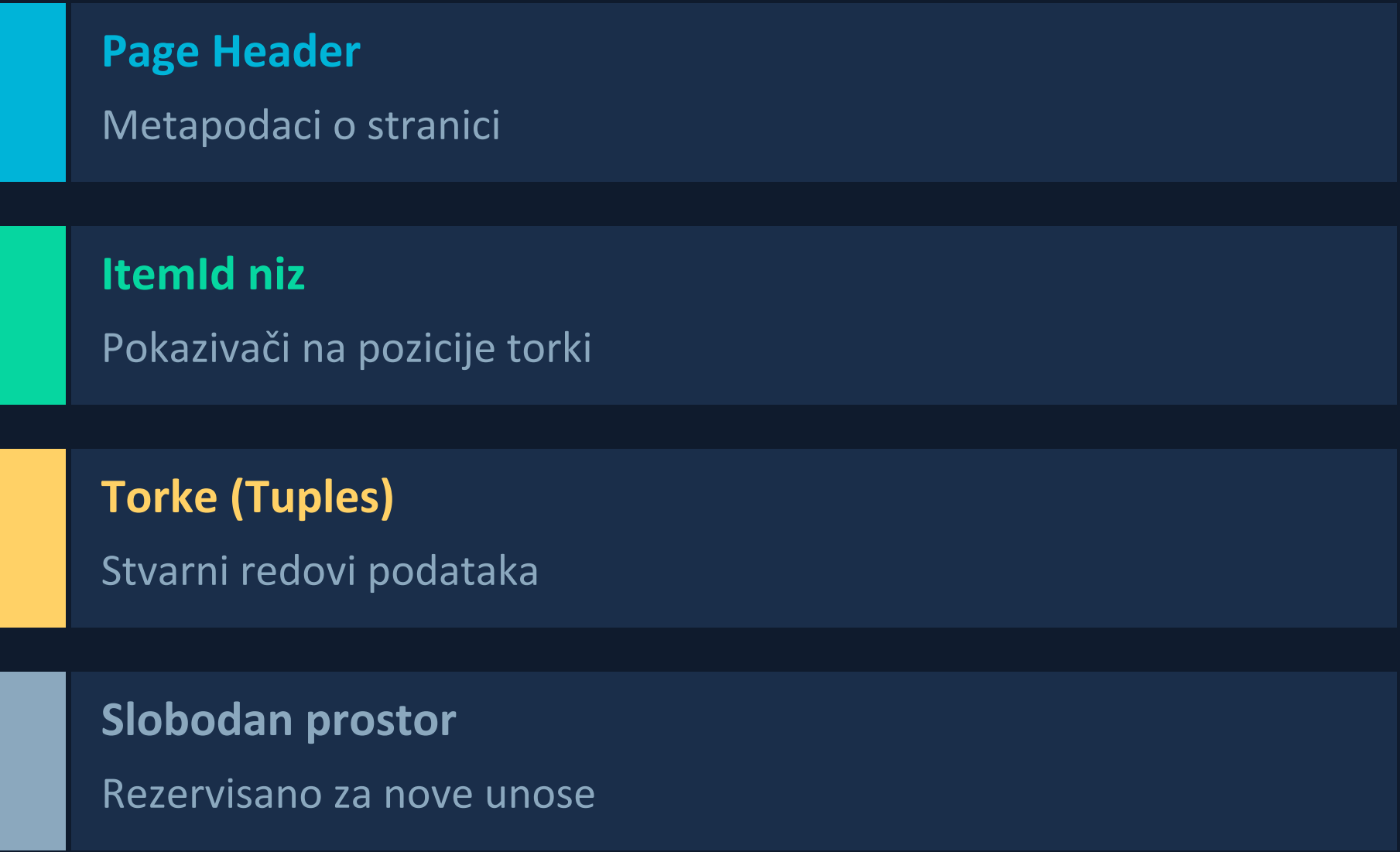
Arhitektura DBMS-a



Heap Storage Model + Stranica (Page)

Heap storage: redovi se smeštaju u prvi slobodan prostor, bez sortiranja. Brzo umetanje, ali potrebni indeksi za efikasnu pretragu.

Anatomija jedne stranice (8 KB):



Svaki disk pristup učitava celu stranicu (8 KB) — nikad manji deo. Zbog toga I/O trošak se meri brojem stranica, ne brojem redova.

Buffer Pool — osnovna memorijska komponenta

Buffer pool (shared_buffers) je memorijski prostor u kome PostgreSQL čuva kopije disk stranica. Sve operacije čitanja i pisanja prolaze kroz ovaj sloj pre nego što dospeju do diska.

Cache Hit (shared hit)

Stranica je u RAM-u → čitanje ~100 ns, bez disk operacije

Cache Miss (shared read)

Stranica nije u RAM-u → učitava se sa diska, ~100 μs–10 ms

Dirty Page

Izmenjena stranica u memoriji koja još nije upisana na disk

Buffer Hit Ratio

$\text{shared_hit} / (\text{shared_hit} + \text{shared_read}) \times 100\%$ — cilj > 95%

Buffer Hit Ratio iz naših testova: 99.2% (cold) → 100% (warm)

Interna organizacija

Page Table

Page ID → Frame

customers/0 → F3

orders/0 → F1

orders/1 → F12

Frame 1

orders/0

dirty=~~X~~ pin=2

Frame 3

customers/0

dirty=✓ pin=1

Frame 12

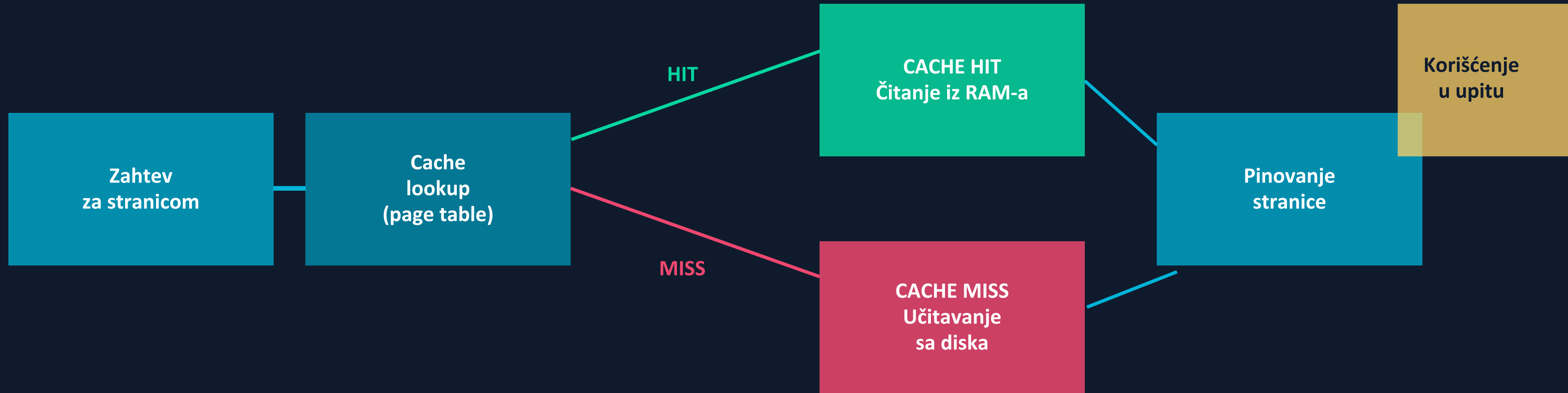
orders/1

dirty=✓ pin=3

Frame 15

FREE

Životni ciklus stranice u Buffer Pool-u



Dirty Marking

Ako je stranica izmenjena → dirty flag = true

Unpinovanje

pin counter - kada executor završi sa stranicom

Evikcija (LRU)

Stranica ostaje ili se uklanja; dirty → flush na disk pre uklanjanja

WAL garancija

Pre upisa dirty page → zapis mora biti u Write-Ahead Log

Cold Cache vs Warm Cache

COLD CACHE

Kada se javlja?

Nakon DISCARD ALL, prvog pokretanja sistema, ili prvog pristupa novim podacima

Šta se dešava?

Svaki zahtev generiše shared read (disk I/O) — stranice se učitavaju sa diska jednu po jednu

Efekat

Visoko vreme izvršavanja, intenzivan disk pristup

WARM CACHE

Kada se javlja?

Nakon što su relevantne stranice učitane u buffer pool prethodnim upitima

Šta se dešava?

Zahtevi se opslužuju kao shared hit — direktno iz RAM-a bez disk operacije

Efekat

Minimalno vreme izvršavanja — čisto CPU procesiranje

LRU algoritam (Least Recently Used)

Kada se buffer pool popuni, PostgreSQL izbacuje stranicu koja se najduže nije koristila. Na većim bazama (> shared_buffers) ovo postaje kritično — stranice koje su upravo učitane mogu biti izbačene pre nego što se ponovo koriste (cache pollution pri seq scan).

Caching mehanizmi — Slojevita memorijska hijerarhija

DBMS sloj keširanja

CPU keš (L1/L2/L3)

Hardverski upravljan, nevidljiv za DBMS. Latencija ~1 ns. Kešira instrukcije i indeksne cvorove.

shared_buffers (Buffer Pool)

Primarni PostgreSQL keš stranica (8 KB). Konfigurira se u postgresql.conf. Preporuka: ~25% RAM-a.

OS Page Cache

Kernel-level keš disk blokova. PostgreSQL ga koristi posredno — rizik dupliranja stranica u memoriji.

Fizički medijumi

SSD / NVMe disk

Latencija ~100 μs. PostgreSQL može koristiti tablespace-ove na SSD-u za vruće tabele i indekse.

HDD disk (magnetni)

Najsporiji nivo, latencija ~10 ms. Sekvencijalni pristup znatno bolji od nasumičnog zbog mehanickih ograničenja.

Princip rada

Nijedan sloj ne funkcioniše nezavisno — podatak prolazi kroz sve nivoe. Svaki sloj koji ga uhvati (kešira) sprečava ponavljanje putovanja ka disku pri sledecem pristupu.

Dvoslojno keširanje u PostgreSQL-u

Stranica u shared_buffers → DBMS je opsluži direktno (najbrže). Stranica samo u OS cache → OS je prosledi, ali PostgreSQL mora da je smesti i u shared_buffers. Nije ni u jednom kešu → fizički disk pristup (najsporije).

Caching mehanizmi — Plan Cache i pg_prewarm

Pored buffer pool-a, PostgreSQL koristi i dopunske mehanizme koji zajedno doprinose ukupnoj efikasnosti keširanja sistema.

Plan Cache

Čuva kompajlirane planove izvršavanja upita (PREPARE/EXECUTE). Eliminiše planning time pri ponavljanju — iz testova: 1.5 ms → 0.1 ms.

Ogranicenje Plan Cache-a

Ne ažurira planove automatski kada se statistike promene (posle ANALYZE ili masovnih izmena).

pg_prewarm

Ekstenzija koja eksplicitno učitava stranice u buffer pool pre korisnickih upita. Rešava cold cache problem posle restarta.

Primer korišćenja

```
SELECT pg_prewarm('customers'); -- cela tabela | SELECT  
pg_prewarm('idx_value'); -- indeks
```

Automatski prewarm

`pg_prewarm.autoprewarm = on`

- Snima sadržaj buffer pool-a pri isključivanju servisa
- Restaurira ga pri sledećem pokretanju

Bez pg_prewarm → Rani upiti sporiji

Sa pg_prewarm → Sistem starta u warm cache stanju

Caching mehanizmi — OLTP vs OLAP workload

OLTP — Transakcioni sistemi

Karakteristike upita

Mali, česti upiti koji pristupaju istim redovima i stranicama. Visoka konkurentnost, kratke transakcije.

Keširanje

Visok hit ratio lako dostižan — radni skup podataka je mali i lako staje u shared_buffers. LRU politika dobro funkcionise.

Preporuka

shared_buffers = 25% RAM. Indeksi na svim kolonama koje se cesto pretražuju. Cilj: hit rate > 99%.

OLAP — Analitički sistemi

Karakteristike upita

Retki, obimni upiti koji skeniraju veliku količinu podataka. Full table scan je čest i nezaobilazan.

Keširanje

Teže postići visok hit ratio — radni skup može biti veći od shared_buffers. Seq scan može izazvati cache pollution i potisnuti korisne stranice (LRU flooding).

Preporuka

Povećati shared_buffers i effective_cache_size. Koristiti particionisanje tabela. PostgreSQL ring buffer strategija štiti keš od seq scan zagadjenja.

Ring Buffer strategija (PostgreSQL)

Kada PostgreSQL detektuje sekvencijalni sken velike tabele, koristi mali kružni bafer od svega 256 KB umesto celog buffer pool-a. Na taj način, stranice koje se čitaju tokom seq scan-a ne istiskuju aktivne stranice iz glavnog keša — OLTP i OLAP upiti mogu koegzistirati bez međusobnog zagađivanja keša.

Velicina tabele vs shared_buffers — kljucni faktor

Kada tabela staje u shared_buffers, disk I/O je minimalan cak i pri prvom upitu. Kada tabela premašuje buffer pool, sistem postaje kriticki zavisn od stanja kesa.

Poređenje iz naših testova

Mala baza (67 MB < 128 MB buffer pool)

disk read $\approx$ 0 pri cold cache | warm lako dostižan | razlika minimalna

Velika baza (287 MB > 128 MB buffer pool)

cold cache: 25.792 disk read, 203 ms | warm cache: 0 disk read, 3 ms | razlika 64×

LRU evikcija postaje vidljiva

Stranice učitane Seq Scan-om mogu biti izbacene pre ponovne upotrebe — cache pollution pri velikim tabelama

shared_buffers preporuke

Pocetna vrednost: 25% RAM-a. Dedicovan server: do 40%. Promena zahteva restart PostgreSQL-a.

64×

brže warm vs cold
(velika baza)

287 MB

tabela > buffer pool
— pravi I/O vidljiv

128 MB

shared_buffers
konfiguracija

Testno okruženje

Manja baza

Tabela: **test_cache**

Redova: **500.000 redova**

Veličina: **67 MB**

shared_buffers: **128 MB**

→ Staje u buffer pool → warm cache lako dostižan

Veća baza

Tabela: **test_cache_large**

Redova: **3.000.000 redova**

Veličina: **287 MB**

shared_buffers: **128 MB**

→ Premašuje buffer pool → pravi disk I/O vidljiv

Kreiranje tabela i testni upiti:

```
CREATE TABLE test_cache (id BIGSERIAL PRIMARY KEY, value INT, payload TEXT);
INSERT INTO test_cache VALUES ... generate_series(1, 500000);
DISCARD ALL;      -- resetovanje keša pre testa
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM test_cache WHERE value = 500;
```

Rezultati — Mala baza (500k redova, 67 MB)

	Seq Scan	Index — cold	Index — warm
Vreme izvršavanja	48.9 ms	1.56 ms	0.68 ms
Stranica čitano	5.682	510	510
Shared hit	5.682	506	510
Shared read (disk)	0	4	0
Plan	Parallel Seq Scan	Bitmap Scan	Bitmap Scan

Seq Scan čita 11× više stranica (5.682 vs 510) — ali baza staje u keš, pa nema disk I/O

Warm cache: 0.68 ms vs cold 1.56 ms — isti plan, samo stanje keša je drugačije

Optimizator ne menja plan između izvršavanja — jedina promena je buffer pool stanje

Uticaj selektivnosti na plan izvršavanja

WHERE value < 200

Selektivnost: ~20% tabele

Plan: Bitmap Index Scan
+ Bitmap Heap Scan

Vreme: 27.3 ms

Shared hit: 5.706

Shared read: 66

→ Indeks isplativ — grupisano čitanje heap blokova smanjuje nasumični I/O

WHERE value < 800

Selektivnost: ~80% tabele

Plan: Seq Scan

Vreme: 42.6 ms

Shared hit: 5.682

Shared read: 0

→ Index bi generisao previše nasumičnih pristupa — Seq Scan je efikasniji

value < 200 (Bitmap) ima više shared read (66) nego value < 800 (Seq Scan, 0). Bitmap pristupa nasumičnim blokovima po celoj tabeli, dok Seq Scan koristi podatke koji su već u kešu iz prethodnih testova.

Rezultati — Velika baza (3M redova, 287 MB)

	Seq Scan	Index — cold	Index — warm
Vreme izvršavanja	203.5 ms	20.2 ms	3.2 ms
Stranica čitano	28.096	2.919	2.919
Shared hit	2.304	287	2.919
Shared read (disk)	25.792	2.637	0
Plan	Parallel Seq Scan	Bitmap Scan	Bitmap Scan

Seq Scan bez indeksa čita 28.096 stranica — od toga 25.792 sa diska (hit rate samo 8%)

Warm cache: 3.2 ms vs cold 20.2 ms — 6.4× brže, disk read pada na 0 (100% hit rate)

Indeks ubrzava 10× čak i pri hladnom kešu (20 ms vs 203 ms) — eliminisanjem 90% stranica

Uticaj selektivnosti — Velika baza (3M redova)

WHERE value < 200

Selektivnost: ~60% tabele

Plan: Bitmap Index Scan
+ Bitmap Heap Scan

Vreme: 243.8 ms

Shared hit: 702

Shared read: 28.367

→ Indeks koristi grupisano citanje, ali je 60% tabele previse — overhead indeksa vidljiv

WHERE value < 800

Selektivnost: ~80% tabele

Plan: Seq Scan

Vreme: 918.4 ms

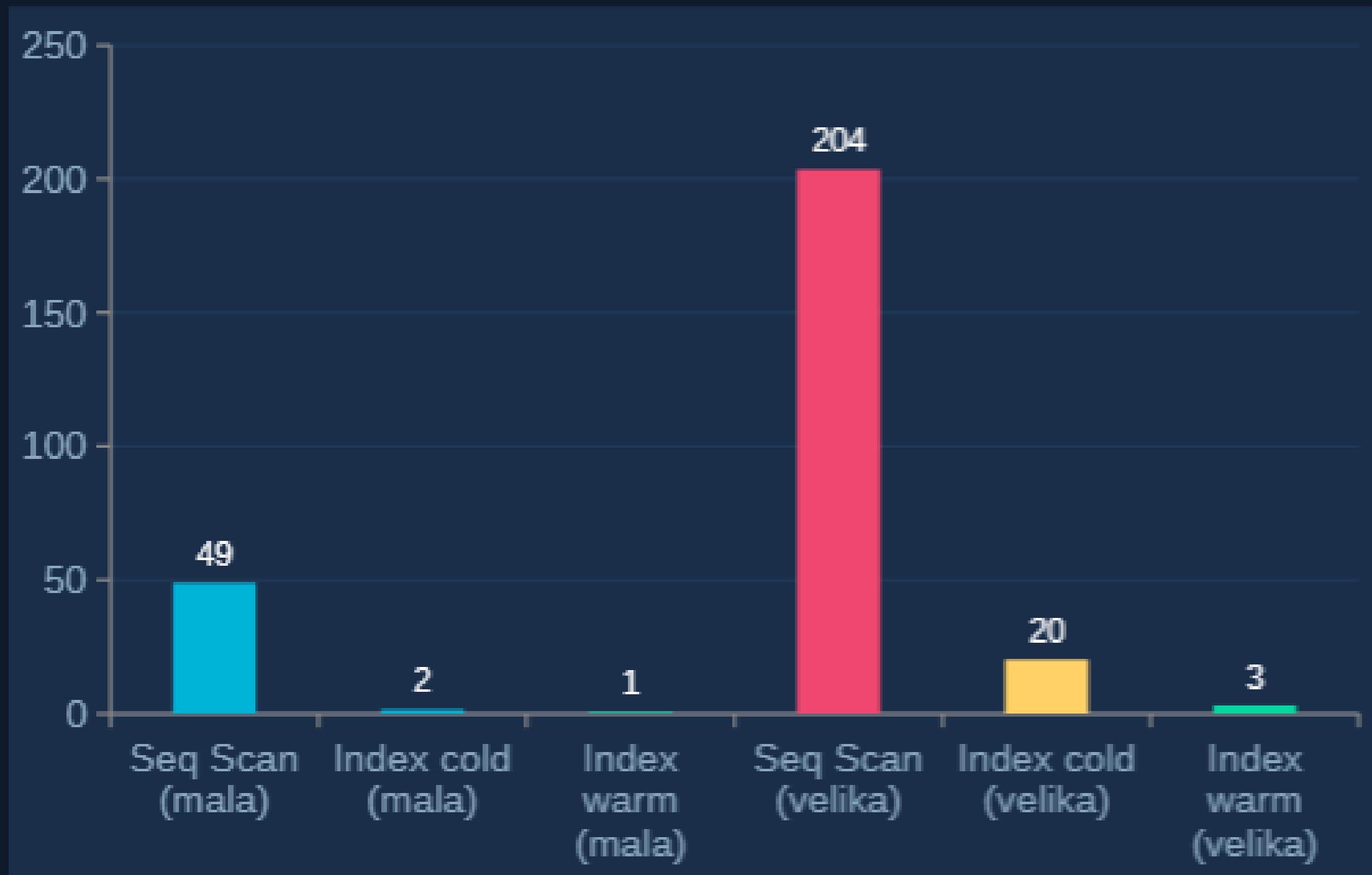
Shared hit: 16.210

Shared read: 11.886

→ Seq Scan ispravno bira optimizer — indeks bi bio sporiji za 80% tabele

Granica efikasnosti indeksa: pri >20% tabele, Bitmap Scan postaje skuplji od Seq Scan. Na velikoj bazi (287MB > shared_buffers), cold cache drasticno pojacava razliku — hit rate pada na 2.5% pri value < 200.

Mala vs Velika baza — kada buffer pool postaje presudan



25.792

disk čitanja
Seq Scan cold (velika)

~64x

brže warm vs cold
(velika baza)

0

disk čitanja
Seq Scan (mala baza)

Kada baza premašuje shared_buffers, buffer pool postaje presudan — cold cache generiše masovan disk I/O, warm cache eliminiše ga potpuno. Razlika: 203 ms → 3 ms.

Zaključak

- 1 Buffer pool eliminiše disk I/O — razlika cold vs warm može biti i 64× (203ms → 3ms na velikoj bazi)
- 2 Indeks smanjuje broj čitanih stranica 11× (5.682 → 510) — direktno smanjujući pritisak na buffer pool
- 3 PostgreSQL optimizer dinamički bira plan — Bitmap Scan za srednju selektivnost, Seq Scan za visoku (>80%)
- 4 Plan ostaje isti pri ponavljanju upita — jedina promena performansi je stanje buffer pool-a (cold vs warm)
- 5 Veličina baze vs shared_buffers je presudna — 67MB staje u keš, 287MB ne → LRU eviction postaje vidljiv
- 6 Buffer pool i optimizer su odvojeni ali komplementarni mehanizmi — optimizer bira plan, buffer pool minimizuje I/O